pandas Essentials

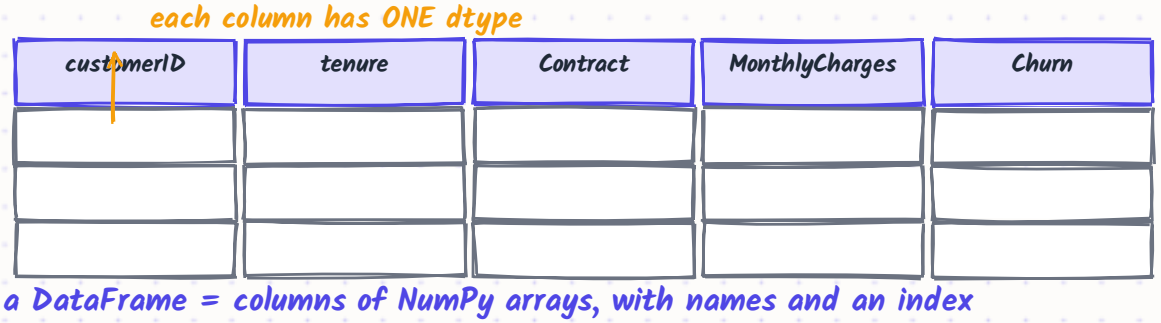
Lesson 4 · Lab 2 · the 12 calls that do 90% of the work

In this lesson → ① ChurnLens ② First look ③ 5 checks ④ Selecting ⑤ groupby ⑥ Summary table ⑦ 3 mistakes ⑧ Interview Q&A

Lab data: ChurnLens, 7,085 telecom customers, messy on purpose — download it from the lesson page.

1 The DataFrame

A **DataFrame** = columns of NumPy arrays (one dtype each) + names + an index.




```
import pandas as pd

df = pd.read_csv("churnlens.csv")
df.shape      # (7085, 15)
df.head()     # first 5 rows
df.info()     # non-null counts + dtypes  ← never skip this
```

2 The five checks — run these on every new file

Check	Code	ChurnLens says
Missing	`df.isna().sum()`	MonthlyCharges 95, TotalCharges 182
Duplicates	`df.duplicated().sum()`	42 exact duplicate rows
Dtypes	`df.dtypes`	TotalCharges is text, not a number
Categories	`df["gender"].value_counts()`	Male / MALE / Female / female
Dates	`pd.to_datetime(s, dayfirst=True)`	SignupDate is a dd/mm/yyyy string

3 Selecting

```
df["tenure"]          # one column
df[["tenure","Churn"]] # several

df.loc[0, "Contract"]  # by LABEL
df.iloc[0, 6]          # by POSITION
```

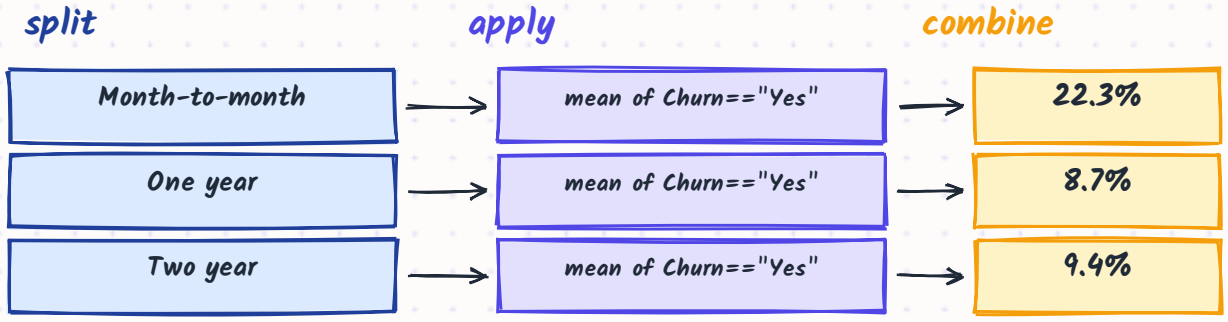
```
m = ((df["Contract"] == "Month-to-month")
   & (df["MonthlyCharges"] > 80))
df[m].shape    # (1298, 15)

df.loc[m, "MonthlyCharges"].mean()
```

Same as NumPy

Boolean masks work exactly as in Lab 1 — brackets around each condition, `&` for and, `|` for or.

4 groupby: split · apply · combine




```
df.groupby("Contract").agg(
   customers   = ("customerID", "size"),
   avg_monthly = ("MonthlyCharges", "mean"),
   churn       = ("Churn", lambda s: (s == "Yes").mean()),
).round(2)
```

Contract	customers	avg monthly	churn rate
Month-to-month	3,851	£64.20	**22.3%**
One year	1,511	£63.38	8.7%
Two year	1,723	£65.47	9.4%

5 The 12 calls that cover 90% of the work

Look	Clean	Reshape / answer
`read_csv`	`isna` / `fillna` / `dropna`	`groupby`
`head` / `sample`	`drop_duplicates`	`agg`
`info` / `dtypes`	`astype` / `to_numeric`	`merge`
`describe`	`str.strip().str.title()`	`value_counts`
`shape`	`to_datetime`	`sort_values`

6 3 mistakes that catch everyone

Mistake	What happens	Fix
Chained assignment `df[mask]["col"] = 0`	writes to a copy — silently does nothing	`df.loc[mask, "col"] = 0`
Trusting `read_csv` dtypes	one blank turns a number column into text	`pd.to_numeric(s, errors="coerce")` then count NaNs
Forgetting the result	`df.drop(columns=[...])` returns a new frame	assign it: `df = df.drop(...)`

ChurnLens

After `to_numeric`, **272** TotalCharges values are NaN — the 182 blanks plus 90 rows where a single space was used. Those are brand-new customers with `tenure = 0`: not missing, but **zero**.

7 Interview corner

Q. loc vs iloc?

A. loc selects by label (index value / column name), iloc by integer position.

Q. What does groupby do?

A. Split rows into groups, apply a function to each group, combine the results into a new frame.

Q. Why would a numeric column load as object?

A. Something in it is not a number — a blank, "-", a comma or a currency symbol. Coerce it and count what fails.

Q. How do you find duplicates?

A. `df.duplicated().sum()` for exact rows; `df.duplicated(subset=["customerID"])` for a key.

Q. apply vs vectorised?

A. apply runs a Python function per row — slow. Prefer column arithmetic, `map`, or `np.where`.

⚡ Revise in 60 seconds

- → read_csv → shape → head → **info**
- → 5 checks: missing · duplicates · dtypes · categories · dates
- → masks filter rows; loc = labels, iloc = positions
- → groupby + agg = the summary table you present
- → chained assignment writes to a copy — use .loc
- → a text column with one blank breaks the dtype

Lab checklist

- ✓ loaded churnlens.csv and read `info()` line by line
- ✓ found the 42 duplicates and the 4 spellings of gender
- ✓ converted TotalCharges and explained the 272 NaNs
- ✓ produced the churn-rate-by-contract table
- ✓ reproduced the chained-assignment trap once, on purpose

Next lesson → DS-05
Lab: **SQL for data scientists** — joins, GROUP BY and window functions
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

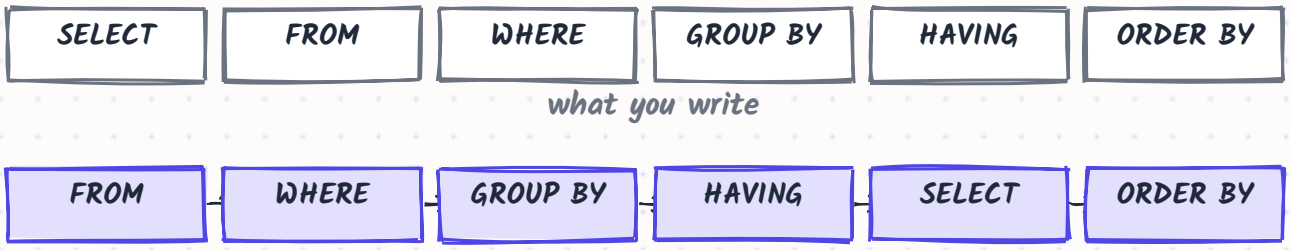
SQL for Data Scientists

Lesson 5 · Lab 3 · joins, GROUP BY and window functions

In this lesson → ① Why SQL ② Run order ③ GROUP BY ④ WHERE vs HAVING ⑤ JOINs ⑥ Windows ⑦ 3 mistakes ⑧ Interview Q&A

Lab data: ChurnLens, now as two SQLite tables — customers and 4,000 support tickets.

1 The order a query actually runs



You write	The database runs	Consequence
SELECT ... FROM ...	1. FROM / JOIN	tables and joins resolve first
WHERE ...	2. WHERE	row filter — cannot see a SELECT alias
GROUP BY ...	3. GROUP BY	rows collapse into groups
HAVING ...	4. HAVING	group filter — can use aggregates
SELECT columns	5. SELECT	aliases are created here
ORDER BY ...	6. ORDER BY	the only place an alias works

Why it matters
This single table explains "unknown column" errors and why a group filter must go in **HAVING**, not WHERE.

DATA SCIENCE NOTEBOOK

2 Aggregate + GROUP BY

```
SELECT Contract,
    COUNT(*)                              AS customers,
    ROUND(AVG(CASE WHEN Churn='Yes' THEN 1.0 ELSE 0 END)*100, 1) AS churn_pct
FROM customers
GROUP BY Contract
ORDER BY churn_pct DESC;
```

Contract	customers	churn_pct
Month-to-month	3,851	**22.3**
Two year	1,723	9.4
One year	1,511	8.7

→ Everything in SELECT that is not inside an aggregate must be in GROUP BY

→ `AVG(CASE WHEN ... THEN 1.0 ELSE 0 END)` is the standard "rate of" trick

→ `COUNT(DISTINCT customerID)` when rows can repeat

3 WHERE vs HAVING

WHERE – before grouping

```
WHERE InternetService = 'Fiber optic'
```

HAVING – after grouping

```
GROUP BY Contract
HAVING COUNT(*) > 2000
```

4 JOINs

Join	Keeps	Use it when
INNER	rows matching in both	you only want customers who have tickets
LEFT	all left rows + matches	**the analysis default** — every customer, tickets if any
RIGHT	all right rows	almost never — swap the tables and use LEFT
FULL	everything, NULLs where missing	reconciling two sources

```
SELECT c.Contract,
    COUNT(t.ticket_id)      AS tickets,
    ROUND(AVG(t.resolved_hours), 1) AS avg_hrs
FROM customers c
LEFT JOIN tickets t ON c.customerID = t.customerID
GROUP BY c.Contract;
-- Month-to-month 2247 12.1 | One year 852 11.8 | Two year 933 12.0
```

Silent bug
An **INNER** JOIN here would drop every customer who never raised a ticket — and every average you report would shift.

5 Window functions — the senior differentiator

```
SELECT customerID, Contract, MonthlyCharges,
    ROW_NUMBER() OVER (PARTITION BY Contract
                ORDER BY MonthlyCharges DESC) AS rn
FROM customers;

-- running total
SUM(amount) OVER (PARTITION BY customerID ORDER BY month)
-- previous month
LAG(amount) OVER (PARTITION BY customerID ORDER BY month)
```

Function	Gives you
`ROW_NUMBER()`	a unique 1,2,3 per partition — top-N per group
`RANK()` / `DENSE_RANK()`	ranking with ties handled two ways
`SUM() OVER (...)`	running totals
`LAG()` / `LEAD()`	previous / next row — month-over-month change

The difference

GROUP BY **collapses** rows. A window function **keeps** every row and adds a number.

Learn these

`ROW_NUMBER · RANK · SUM() OVER · LAG` — four functions that cover most window-function questions.

7 Interview corner

Q. WHERE vs HAVING?

A. WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q. What does LEFT JOIN do?

A. Keeps every row from the left table; columns from the right are NULL where nothing matched.

Q. Name a window function and what it is for.

A. ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...) — top-N per group without collapsing rows.

Q. Find the second highest value.

A. `DENSE_RANK() OVER (ORDER BY x DESC) = 2`, or a subquery with MAX excluding the max.

Q. How do you find duplicates?

A. `GROUP BY key HAVING COUNT(*) > 1`.

⚡ Revise in 60 seconds

- → run order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- → WHERE = rows, HAVING = groups
- → LEFT JOIN is the analysis default
- → window functions keep rows, GROUP BY collapses them
- → COUNT(col) skips NULLs
- → watch integer division

Next lesson → DS-06
Reading a dataset for the **first time** — the 10-step EDA checklist
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

Reading a Dataset for the First Time

Lesson 6 · the 10-step EDA checklist

In this lesson → ① The blank page ② Shape + target ③ Missingness ④ Distributions ⑤ Outliers ⑥ Relationships ⑦ Correlation traps ⑧ Interview Q&A

Module dataset: ChurnLens, 7,085 telecom customers — 16.3% of them churn.

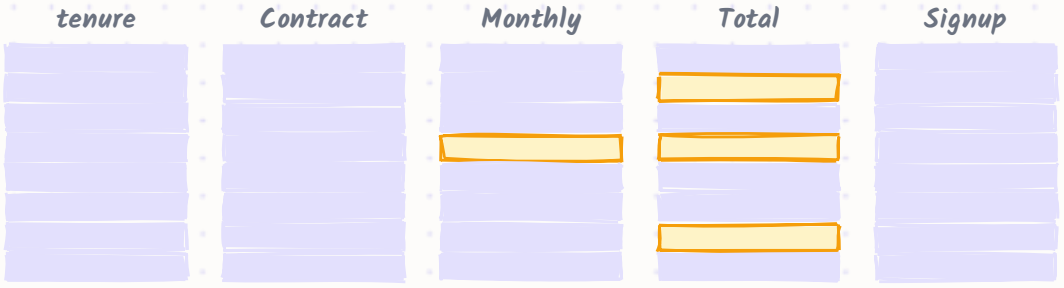
1 The 10-step first look

> EDA is a **fixed checklist**, run the same way every time — not "make some charts".

#	Step	One line of code
1	Shape — is it what you expected?	`df.shape`
2	Target — what share is positive?	`df[y].value_counts(normalize=True)`
3	Types — numbers, categories, dates	`df.dtypes`
4	Missingness — where are the holes?	`df.isna().mean().sort_values()`
5	Distributions — one per numeric column	`df.hist(figsize=(12,8))`
6	Outliers — flagged, not deleted	`df.describe()` + box plots
7	Numeric vs target	`df.groupby(y)[col].median()`
8	Categorical vs target	`df.groupby(col)[y].mean()`
9	Correlations — and multicollinearity	`df.corr(numeric_only=True)`
10	Write down 5 findings	markdown cell at the top

ChurnLens
Target is **16.3% churn** → a model that predicts "nobody churns" is already 83.7% accurate. That is the number to beat.

2 Missingness — the step people rush



a missingness map: are the gaps random, or all in one group?

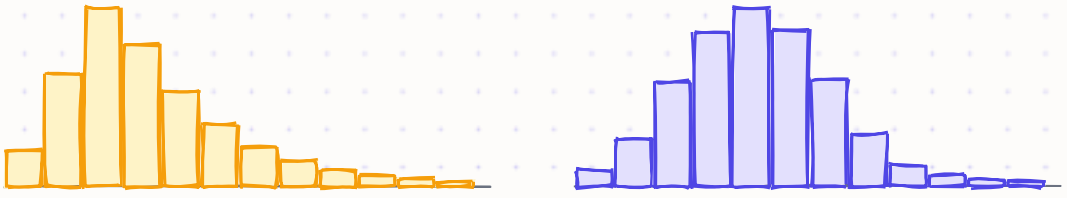
→ Count them: `df.isna().mean().sort_values(ascending=False)`

→ Then **look at where they sit** — random, or all in one group?

→ Missing because of something is information: add an `is_missing` flag before you impute

→ Dropping rows can quietly delete exactly the customers you care about

3 Distributions



What you see	What it usually means
Long right tail	money or counts — try `np.log1p`
Spike at 0	two populations mixed (never-used vs used)
A wall at the maximum	the value is capped or truncated
One round number, thousands of times	a default, not a measurement

4 Outliers



Flag them

```
q1, q3 = df[c].quantile([.25, .75])
iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
df[(df[c] < lo) | (df[c] > hi)]
```

Then decide

- → **Impossible** (negative age) → fix or drop
- → **Extreme but real** → keep it
- → In fraud / failure / churn the outliers are the signal
- → Or use models that tolerate them (trees)

5 Relationships with the target

```
# categorical → rate per category
df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean())
# Month-to-month 0.223 | One year 0.087 | Two year 0.094

# numeric → compare the two groups
df.groupby("Churn")["tenure"].median()  # stayed 40 · left 17
```

This is feature selection by eye
Tenure, contract and payment method separate strongly. Gender does not. You learn this before any model exists.

7 Write it down — 5 bullets, at the top

- What this data is, in one sentence
- What is broken (and what I did about it)
- What surprised me
- What I assumed
- What I need to ask the data owner

6 Correlation traps

Trap	Why it bites
Correlation = importance	a feature can matter only *in combination* with another
Zero correlation = no relationship	Pearson only sees straight lines — a U-shape scores ~0
Ignoring feature-to-feature correlation	tenure × TotalCharges are near-duplicates (total = monthly × tenure) → unstable, meaningless coefficients
Correlation on raw skewed data	one extreme row can make or break it

8 Interview corner

Q. How would you explore a dataset you have never seen?

A. Shape → target → types → missingness → distributions → outliers → relationships → correlations → write down five findings. Same order every time.

Q. When do you remove an outlier?

A. Only when the value is impossible. Extreme-but-real values are often the signal — especially in churn, fraud and failure data.

Q. Zero correlation with the target — drop it?

A. No. Pearson sees only straight lines; check a non-linear relationship and tree importance.

⚡ Revise in 60 seconds

- → run the same 10 steps every time
- → the target rate sets your baseline (83.7% here)
- → look at where the missing values sit, not just how many
- → skew → log; spike at 0 → two populations
- → delete only impossible values
- → correlation ≠ importance; watch feature-to-feature

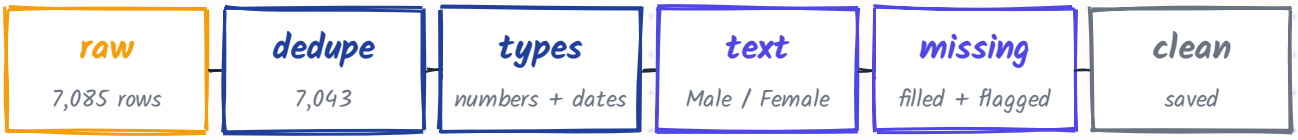
Next lesson → DS-07
Lab: **data cleaning** — missing values, duplicates and outliers, done properly
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

Data Cleaning, Done Properly

Lesson 7 · Lab 4 · missing values, duplicates, outliers

In this lesson → ① The mess list ② Order ③ Duplicates ④ Types + text ⑤ 3 kinds of missing ⑥ Filling ⑦ Outliers ⑧ Interview Q&A
Lab data: ChurnLens, 7,085 raw rows in, 7,043 clean rows out — in one rerunnable function.

1 Clean in this order



one function, run top to bottom — never hand-edits scattered through a notebook

#	Step	Why it comes first
1	Duplicates	every later statistic depends on the row count
2	Types	you cannot check a number stored as text
3	Text / categories	value_counts is meaningless while "MALE" ≠ "Male"
4	Missing values	needs correct types and real rows first
5	Outliers	last — and often the answer is "none"

The real rule
Cleaning lives in **one function**, not in scattered notebook cells. Reviewable, rerunnable, and you can say exactly what changed.

2 Duplicates

```
df = df.drop_duplicates()      # 7085 → 7043  (42 exact rows)

# the dangerous kind: same key, different values
df["customerID"].duplicated().sum()
df.sort_values("SignupDate").drop_duplicates("customerID", keep="last")
```

3 Types and text

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")  # 270 → NaN
df["SignupDate"]   = pd.to_datetime(df["SignupDate"], dayfirst=True)    # 2019-01-01 → 2023-12-05

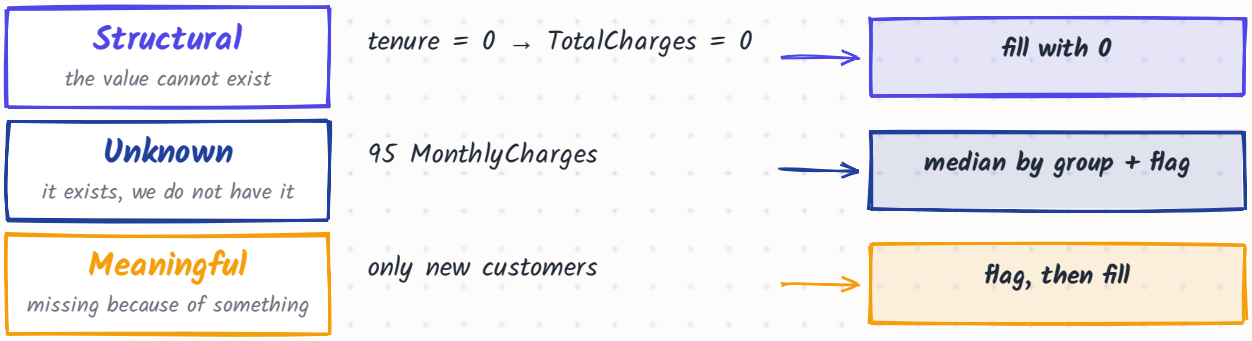
df["gender"] = df["gender"].str.strip().str.title()    # Male 3548 · Female 3495
df["TechSupport"] = df["TechSupport"].replace("No internet service", "No")  # 60 rows
```

→ `errors="coerce"` then **count the NaNs it created** — that number is a finding

→ `.str.strip().str.title()` fixes most category chaos

→ Fold categories that mean the same thing, but write down that you did

4 Missing values are three different problems



Kind	Meaning	ChurnLens	Do this
Structural	the value cannot exist	92 rows: tenure = 0 → no total charges	fill with **0**
Unknown	it exists, we do not have it	178 TotalCharges, 95 MonthlyCharges	median **within a group** + flag
Meaningful	missing because of something	e.g. only new customers	the flag is a **feature**

5 Filling properly

```
# 1. flag BEFORE you fill
df["charges_missing"] = df["MonthlyCharges"].isna()

# 2. structural → 0
df.loc[df.tenure == 0, "TotalCharges"] = 0

# 3. unknown → median within a sensible group
df["MonthlyCharges"] = (df.groupby("InternetService")["MonthlyCharges"]
                        .transform(lambda s: s.fillna(s.median())))
```

Group	Median monthly charge
No internet	£21.06
DSL	£57.77
Fiber optic	£91.11
overall	£66.17 — would invent customers who do not exist

Never
Do not fill everything with the overall mean. It shrinks variance, breaks group differences, and hides what you changed.

6 Outliers — and the honest answer

```
q1, q3 = df["MonthlyCharges"].quantile([.25, .75])
iqr    = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr     # -30.07 … 160.40
df[(df.MonthlyCharges < lo) | (df.MonthlyCharges > hi)].shape   # (0, 15)
```

Finding
No outliers in MonthlyCharges. That is a result — report it. Do not invent an outlier step because a tutorial had one.

Q. How do you handle missing values?

A. Ask why first. Structural → 0. Unknown → median within a group + a flag. Meaningful → the flag is a feature. Never "the mean" as a reflex.

Q. Would you drop rows with missing values?

A. Only if few and random — dropping can delete exactly the group you were asked about.

7 The cleaning function

```
def clean(raw: pd.DataFrame) -> pd.DataFrame:
    df = raw.drop_duplicates().copy()
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    df["SignupDate"]   = pd.to_datetime(df["SignupDate"], dayfirst=True)
    df["gender"]       = df["gender"].str.strip().str.title()
    df["TechSupport"]  = df["TechSupport"].replace("No internet service", "No")
    df["charges_missing"] = df["MonthlyCharges"].isna()
    df.loc[df.tenure == 0, "TotalCharges"] = 0
    df["MonthlyCharges"] = (df.groupby("InternetService")["MonthlyCharges"]
                            .transform(lambda s: s.fillna(s.median())))
    print(len(raw), " → ", len(df))
    return df

clean_df = clean(pd.read_csv("churnlens.csv"))
clean_df.to_parquet("churnlens_clean.parquet")
```

8 Interview corner

Q. Mean or median?

A. Median for skewed data — the mean is dragged by the tail. Impute within a group where one exists.

Q. How do you make cleaning reproducible?

A. One function raw → clean, row counts printed, raw file never overwritten.

⚡ Revise in 60 seconds

- → duplicates → types → text → missing → outliers
- → 3 kinds of missing: structural · unknown · meaningful
- → flag before you fill; median within a group
- → "no outliers" is a valid result

Next lesson → DS-08
Descriptive statistics — which number actually describes your data
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**